

Menu Driven Program on Array Operations

Name: Rudra Pratap Singh

SAP ID: 590032811

Batch: 16

1. Aim / Objective

To write a menu driven C program that performs the following operations on a one-dimensional array: (a) sorting the array using the Bubble Sort algorithm and searching an element using Binary Search, (b) inserting and deleting an element at a user-specified valid position, and (c) rotating the array elements to the right by k positions.

2. Menu Options

Choice	Operation
1	Sort the array using Bubble Sort and perform a Binary Search
2	Perform Insertion and Deletion of an element at a valid position
3	Rotate the array elements to the right by k positions
4	Exit the program

3. Algorithm

3.1 Bubble Sort

- Step 1: Start with the first element of the array.
- Step 2: Compare each pair of adjacent elements; if the left element is greater than the right, swap them.
- Step 3: Repeat the pass over the array, reducing the range by one each time, until no swaps occur.
- Step 4: The array is now sorted in ascending order.

3.2 Binary Search

- Step 1: The array must already be sorted (done via Bubble Sort).
- Step 2: Set low = 0 and high = n - 1.
- Step 3: Find mid = (low + high) / 2 and compare arr[mid] with the key.
- Step 4: If equal, the element is found at index mid. If key is greater, search the right half (low = mid + 1); otherwise search the left half (high = mid - 1).
- Step 5: Repeat until the element is found or low > high (element not present).

3.3 Insertion at a Given Position

- Step 1: Check whether the entered position is valid (between 1 and n + 1).
- Step 2: Shift all elements from the last index up to the given position one place to the right.
- Step 3: Place the new value at the vacated position and increase the array size (n) by 1.

3.4 Deletion at a Given Position

- Step 1: Check whether the entered position is valid (between 1 and n).
- Step 2: Shift all elements after the given position one place to the left, overwriting the deleted element.
- Step 3: Decrease the array size (n) by 1.

3.5 Right Rotation by k Positions

- Step 1: Reduce k using $k = k \% n$, so that rotations larger than the array size are handled correctly.
- Step 2: Copy every element arr[i] into a temporary array at position (i + k) % n.
- Step 3: Copy the temporary array back into the original array.

4. Complexity Analysis

Operation	Time Complexity	Space Complexity
Bubble Sort	$O(n^2)$ worst case, $O(n)$ best case	$O(1)$
Binary Search	$O(\log n)$	$O(1)$
Insertion at position	$O(n)$ worst case	$O(1)$
Deletion at position	$O(n)$ worst case	$O(1)$
Right Rotation by k	$O(n)$	$O(n)$ (temporary array)

5. Source Code (C Program)

File name: array_operations.c

```
/*
 * =====
 * Program : Menu Driven Array Operations
 * Description:
 * This program performs the following operations on an
 * array using a menu driven approach:
 * 1. Sort the elements using Bubble Sort and
 * perform Binary Search on the sorted array.
 * 2. Perform Insertion and Deletion at a given
 * valid position.
 * 3. Rotate the array elements to the right by
 * k positions.
 * 4. Exit
 * =====
 */

#include <stdio.h>

#define MAX 100 /* Maximum possible size of the array */

/* ----- Function Prototypes ----- */
void displayArray(int arr[], int n);
void bubbleSort(int arr[], int n);
int binarySearch(int arr[], int n, int key);
void insertElement(int arr[], int *n, int pos, int value);
void deleteElement(int arr[], int *n, int pos);
void rotateRight(int arr[], int n, int k);

int main()
{
    int arr[MAX], n, choice, key, index;

    /* ----- Read the original array from the user ----- */
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    printf("Enter %d elements of the array:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    /* ----- Menu Driven Loop ----- */
    do
    {
        printf("\n-----\n");
        printf("MAIN MENU\n");
        printf("-----\n");
        printf("1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion at a given position\n");
        printf("3. Rotate array to the right by k positions\n");
        printf("4. Exit\n");
        printf("-----\n");
```

```

printf("Enter your choice (1-4): ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        /* ----- Bubble Sort ----- */
        bubbleSort(arr, n);
        printf("\nArray after Bubble Sort:\n");
        displayArray(arr, n);

        /* ----- Binary Search ----- */
        printf("\nEnter the element to search: ");
        scanf("%d", &key);

        index = binarySearch(arr, n, key);

        if (index != -1)
            printf("Element %d found at position %d (index %d).\n",
                key, index + 1, index);
        else
            printf("Element %d not found in the array.\n", key);
        break;

    case 2:
    {
        /* ----- Insertion ----- */
        int insChoice;
        printf("\n--- Insertion / Deletion Menu ---\n");
        printf("1. Insert an element\n");
        printf("2. Delete an element\n");
        printf("Enter your choice (1-2): ");
        scanf("%d", &insChoice);

        if (insChoice == 1)
        {
            int pos, value;

            if (n >= MAX)
            {
                printf("Array is full. Cannot insert more elements.\n");
                break;
            }

            printf("Enter the position (1 to %d) to insert at: ", n + 1);
            scanf("%d", &pos);

            if (pos < 1 || pos > n + 1)
            {
                printf("Invalid position! Insertion failed.\n");
                break;
            }

            printf("Enter the value to insert: ");
            scanf("%d", &value);

            insertElement(arr, &n, pos, value);
            printf("\nArray after Insertion:\n");
            displayArray(arr, n);
        }
        else if (insChoice == 2)
        {
            int pos;

            if (n == 0)
            {
                printf("Array is empty. Cannot delete.\n");
                break;
            }

```

```

    }

    printf("Enter the position (1 to %d) to delete: ", n);
    scanf("%d", &pos);

    if (pos < 1 || pos > n)
    {
        printf("Invalid position! Deletion failed.\n");
        break;
    }

    deleteElement(arr, &n, pos);
    printf("\nArray after Deletion:\n");
    displayArray(arr, n);
}
else
{
    printf("Invalid choice!\n");
}
break;
}

case 3:
{
    /* ----- Rotation ----- */
    int k;
    printf("\nEnter the number of positions (k) to rotate right: ");
    scanf("%d", &k);

    rotateRight(arr, n, k);
    printf("\nArray after Right Rotation by %d position(s):\n", k);
    displayArray(arr, n);
    break;
}

case 4:
    printf("\nExiting the program. Thank you!\n");
    break;

default:
    printf("\nInvalid choice! Please enter a number between 1 and 4.\n");
}

} while (choice != 4);

return 0;
}

/* =====
 * Function : displayArray
 * Purpose  : Displays the current elements of the array
 * ===== */
void displayArray(int arr[], int n)
{
    if (n == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("[ ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("]\n");
}

/* =====
 * Function : bubbleSort

```

```

* Purpose : Sorts the array in ascending order using the
*           Bubble Sort algorithm
* ===== */
void bubbleSort(int arr[], int n)
{
    int temp, swapped;

    for (int i = 0; i < n - 1; i++)
    {
        swapped = 0;
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                /* Swap arr[j] and arr[j+1] */
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        /* If no elements were swapped, array is already sorted */
        if (swapped == 0)
            break;
    }
}

/* =====
* Function : binarySearch
* Purpose : Searches for "key" in a SORTED array using the
*           Binary Search algorithm.
* Returns : Index of key if found, otherwise -1
* ===== */
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while (low <= high)
    {
        mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1; /* Element not found */
}

/* =====
* Function : insertElement
* Purpose : Inserts "value" at position "pos" (1-based) in
*           the array. Updates the size "n" through pointer.
* ===== */
void insertElement(int arr[], int *n, int pos, int value)
{
    /* Shift elements to the right to make space */
    for (int i = *n; i >= pos; i--)
        arr[i] = arr[i - 1];

    arr[pos - 1] = value;
    (*n)++;
}

/* =====

```

```

* Function : deleteElement
* Purpose  : Deletes the element at position "pos" (1-based)
*           from the array. Updates the size "n" through
*           pointer.
* ===== */
void deleteElement(int arr[], int *n, int pos)
{
    /* Shift elements to the left to fill the gap */
    for (int i = pos - 1; i < *n - 1; i++)
        arr[i] = arr[i + 1];

    (*n)--;
}

/* =====
* Function : rotateRight
* Purpose  : Rotates the array to the right by k positions.
*           Handles k greater than n using the modulus
*           operator.
* ===== */
void rotateRight(int arr[], int n, int k)
{
    int temp[MAX];

    if (n == 0)
        return;

    k = k % n;          /* Handle k >= n */
    if (k < 0)
        k += n;         /* Handle negative k, if any */

    /* Place last k elements at the front, rest after them */
    for (int i = 0; i < n; i++)
        temp[(i + k) % n] = arr[i];

    for (int i = 0; i < n; i++)
        arr[i] = temp[i];
}

```

6. Sample Output

6.1 Option 1 — Bubble Sort followed by Binary Search

Input array: 5 3 8 1 9 2 | Search key: 8

Enter the number of elements in the array: Enter 6 elements of the array:

```
-----
                        MAIN MENU
-----
1. Bubble Sort and Binary Search
2. Insertion and Deletion at a given position
3. Rotate array to the right by k positions
4. Exit
-----
Enter your choice (1-4):
Array after Bubble Sort:
[ 1 2 3 5 8 9 ]
```

Enter the element to search: Element 8 found at position 5 (index 4).

```
-----
                        MAIN MENU
-----
1. Bubble Sort and Binary Search
2. Insertion and Deletion at a given position
3. Rotate array to the right by k positions
4. Exit
-----
Enter your choice (1-4):
Exiting the program. Thank you!
```

6.2 Options 2 and 3 — Insertion, Deletion and Rotation

Input array: 1 2 3 4 5 | Insert 100 at position 3, then delete position 2, then rotate right by 2

Enter the number of elements in the array: Enter 5 elements of the array:

```
-----
                        MAIN MENU
-----
1. Bubble Sort and Binary Search
2. Insertion and Deletion at a given position
3. Rotate array to the right by k positions
4. Exit
-----
Enter your choice (1-4):
--- Insertion / Deletion Menu ---
1. Insert an element
2. Delete an element
Enter your choice (1-2): Enter the position (1 to 6) to insert at: Enter the value to insert:
Array after Insertion:
[ 1 2 100 3 4 5 ]
```

```
-----
                        MAIN MENU
-----
1. Bubble Sort and Binary Search
2. Insertion and Deletion at a given position
3. Rotate array to the right by k positions
4. Exit
-----
Enter your choice (1-4):
--- Insertion / Deletion Menu ---
1. Insert an element
2. Delete an element
```

Enter your choice (1-2): Enter the position (1 to 6) to delete:
Array after Deletion:
[1 100 3 4 5]

MAIN MENU

1. Bubble Sort and Binary Search
 2. Insertion and Deletion at a given position
 3. Rotate array to the right by k positions
 4. Exit
-

Enter your choice (1-4):
Enter the number of positions (k) to rotate right:
Array after Right Rotation by 2 position(s):
[4 5 1 100 3]

MAIN MENU

1. Bubble Sort and Binary Search
 2. Insertion and Deletion at a given position
 3. Rotate array to the right by k positions
 4. Exit
-

Enter your choice (1-4):
Exiting the program. Thank you!

7. Conclusion

The program was successfully designed, implemented and tested. All menu options — Bubble Sort with Binary Search, Insertion and Deletion at a valid position, and Right Rotation by k positions — were verified with sample inputs and produced the correct output in every case. The program correctly validates user input for positions and handles edge cases such as an empty array, a full array, and a rotation count larger than the array size.